

SPRAWOZDANIE SZTUCZNA INTELIGENCJA LAB1

Na sztucznej inteligencji zajmowaliśmy się problemem N-hetmanów.

Poniżej przedstawiam zaimplementowane funkcje dla N-hetmanów.

Funkcja generate_board1

```
def generate_board1(n,struct):  
    if(struct=="board_tuple"):  
        board_tuple=[(random.randint(a: 0, n - 1), random.randint(a: 0, n - 1)) for _ in range(n)]  
        return board_tuple  
    elif(struct=="board_array"):  
        board_array = [[0] * n for _ in range(n)]  
        board_help = [random.randint(a: 0, n - 1) for _ in range(n)]  
        for i in range(n):  
            board_array[board_help[i]][i] = 1  
        return board_array  
    elif(struct=="board_vector"):  
        board_vector=[random.randint(a: 0, n - 1) for _ in range(n)]  
        return(board_vector)  
    elif(struct=="board_perm"):  
        board_perm=list(range(n))  
        random.shuffle(board_perm)  
        return(board_perm)
```

generate_board1 zwraca losową konfigurację N hetmanów w jednej z czterech reprezentacji w zależności od podanego parametru

Board_tuple – lista par opisujących położenie hetmanów w postaci (wiersz, kolumna)

Board_array = tablica dwuwymiarowa n x n, wartość 1 oznacza hetmana, a 0 brak hetmana

Board_vector = tablica jednowymiarowa o długości n. Indeks tablicy oznacza kolumnę, a wartość pod tym indeksem oznacza numer wiersza hetmana. Taka reprezentacja gwarantuje, że w każdej kolumnie jest najwyżej jeden hetman z powodu braku powtórzeń kolumn, co przyspiesza sprawdzanie konfliktu

Board_perm – podobna do board_vector, ale dodatkowo wartości tablicy nie mogą się powtarzać. Co powoduje, że w każdej kolumnie i wierszu występuje tylko jeden hetman. Jest to najbardziej optymalna struktura pod względem późniejszego sprawdzania poprawności ustawienia.

Funkcja Conflicts

```
def conflicts(board):
    my_array=np.array(board)
    n=my_array.shape[0]
    help_array = [-1] * n
    if my_array.ndim == 2 and my_array.shape[1]==2:
        for r, c in my_array:
            if help_array[r] != -1 or c in help_array:
                return False
            help_array[r] = c
    elif my_array.ndim == 2:
        for r in range(n):
            for c in range(n):
                if my_array[r][c] == 1:
                    if help_array[r] != -1 or c in help_array:
                        return False
                    help_array[r]=c
    elif(len(my_array) !=len(set(my_array))):
        return False
    if -1 in help_array:
        help_array=my_array
    for i in range(n):
        for j in range(i+1,n):
            if abs(help_array[i]-help_array[j]) == abs(i-j):
                return False
    return True
```

Funkcja sprawdza czy następuje bicie między hetmanami jest uniwersalna dla czterech struktur. Uznałem, że najszybszym i uniwersalnym sposobem jest doprowadzenie każdej struktury do postaci zgodnej z reprezentacją board_perm która sprawdza czy zachodzą relacje

$$Q_i \leftrightarrow Q_j \Leftrightarrow r_i = r_j \vee c_i = c_j$$

Następnie sprawdzamy czy zachodzą bicia na przekątnej dzięki czemu sprawdzamy występowania wszystkich warunków

$$Q_i \leftrightarrow Q_j \Leftrightarrow r_i = r_j \vee c_i = c_j \vee |r_i - r_j| = |c_i - c_j|.$$

Jak działa funkcja?

Najpierw zamieniamy wejściową strukturę board na tablicę typu numpy dzięki czemu możemy łatwo wyznaczyć n czyli liczbę hetmanów. Następnie tworzymy tablicę pomocniczą help_array wypełnioną wartościami -1 ta tablica służy do zbudowania reprezentacji w stylu board_perm.

Sprawdzamy czy board ma postać board_tuple czyli czy jest tablicą dwuwymiarową z dwoma kolumnami jeśli tak to na podstawie tych par wypełniamy help_array jeśli w tym procesie znajdziemy dwóch hetmanów w tym samym wierszu lub w tej samej kolumnie zwracamy false

Jeśli board jest tablicą dwuwymiarową n x n typu board_array wyszukujemy jedynki w macierzy. Analogicznie do pierwszej struktury sprawdzamy czy tablica spełnia warunki dla reprezentacji vector_perm

Dla struktury jednowymiarowej jak board_vector sprawdzamy czy występują powtórzenia wartości jeśli tak zwracamy false ponieważ oznacza to dwóch hetmanów w tym samym wierszu jeśli nie ma powtórzeń traktujemy tablicę jak board_perm

Na końcu sprawdzamy czy występują bicia po przekątnych porównując wartości $|r_i - r_j|$ i $|c_i - c_j|$ jeśli żaden z warunków nie jest spełniony zwracamy true

Fukcja measure_time

```
def measure_time(N,struct):
    iteration=100
    is_True=0
    total_time=0
    for i in range(iteration):
        start=time.perf_counter()
        board=generate_board1(N,struct)
        is_conflict= conflicts(board)
        stop=time.perf_counter()
        elapsed = stop - start
        total_time += elapsed
        if is_conflict:
            is_True+=1
    return is_True/iteration, total_time/iteration
```

Funkcja oblicza średni czas i średnią liczbę trafień przy stu iteracja czas mierzymy od momentu generowania tablicy do sprawdzenia konfliktów.

Funkcja Experiment

```
def experiment():
    Ns=[4,8,16,32,64,128]
    representations=["board_tuple","board_array","board_vector","board_perm"]
    with open('result.csv', 'w', newline='') as csvfile:
        writer = csv.writer(csvfile)
        writer.writerow(["N","representation_type","execution_time","conflict_rate"])
        for struct in representations:
            for N in Ns:
                avg_hit, avg_time= measure_time(N,struct)
                writer.writerow([N, struct, format(avg_time,".10f"), avg_hit])
```

Funkcja sprawdza dla każdej reprezentacji i różnej ilości hetmanów czasy i liczbę trafień rezultaty zapisuję w pliku result.csv

Przetestowanie funkcji

```
74
75 board_tuple = [(0, 2), (1, 0), (2, 3), (3, 1)]
76 board_array1 = [[0, 0, 1, 0], [1, 0, 0, 0], [0, 0, 0, 1], [0, 1, 0, 0]]
77 board_vector = [2, 0, 3, 1]
78 board_perm = (2, 0, 3, 1)
79
80 tab_struct=[board_tuple, board_array1, board_vector, board_perm]
81 for struct in tab_struct:
82     print(conflicts(struct))
```

Run lab1 x

↑ /Users/mizula/Sztuczna_inteligencja/lab1_ma55831/.venv/bin/python /Users/mizula/S
↓ True
⇌ True
⇕ True
⇓ True

Przetestowałem prezentacje z gotowym ułożeniami hetmanów z pliku laboratoryjnego. Wyniki wyszły poprawne. W żadnej strukturze hetmani się nie biją.

Test wszystkie funkcje wyniki

result			
N	representation_type	execution_time	conflict_rate
4	board_tuple	0.0000035829	100.0
8	board_tuple	0.0000059787	100.0
16	board_tuple	0.0000108621	100.0
32	board_tuple	0.0000219313	100.0
64	board_tuple	0.0000428396	100.0
128	board_tuple	0.0000908566	100.0
4	board_array	0.0000031021	100.0
8	board_array	0.0000065404	100.0
16	board_array	0.0000148012	100.0
32	board_array	0.0000366354	100.0
64	board_array	0.0001105638	100.0
128	board_array	0.0003763875	100.0
4	board_vector	0.0000017338	99.99
8	board_vector	0.0000027434	100.0
16	board_vector	0.0000047266	100.0
32	board_vector	0.0000089687	100.0
64	board_vector	0.0000172911	100.0
128	board_vector	0.0000340050	100.0
4	board_perm	0.0000016917	99.91
8	board_perm	0.0000026012	99.99
16	board_perm	0.0000043854	100.0
32	board_perm	0.0000081191	100.0
64	board_perm	0.0000149654	100.0
128	board_perm	0.0000299788	100.0

Poniżej załączam wyniki jak widać board_array poradziło sobie najgorzej tuż za nim board_tuple. Bardzo dobrze poradziło sobie czasowo board_vector. Najlepiej board_perm

Wnioski koncowe:

-Czytelność kodu

Funkcja Conflicts mogłaby być bardziej czytelna gdyby przyjmowała nazwę reprezentacji i na tej podstawie sprawdzała, z jakim typem ma do czynienia. Jednak chciałem funkcję zrobić bardziej uniwersalna i bez niepotrzebnych zmiennych, stawiałem na prostotę kodu.

-Złożoność obliczeniowa i czas wykonania

Uważam, że złożoność i czas wykonania funkcji są bardzo dobre, ponieważ każdą strukturę w conflicts sprowadzam do najbardziej optymalnej reprezentacji board_perm. Dzięki temu sprawdzanie konfliktów jest szybkie. Jednak występuje niepotrzebne sprawdzanie, czy reprezentacja board_perm zawiera duplikaty. Aby to wyeliminować, funkcja mogłaby przyjmować nazwę reprezentacji i w przypadku board_perm pomijać zbędne warunki (np. sprawdzanie powtórzeń wierszy). Mimo to reprezentacja board_perm pozostaje najszybsza, ponieważ warunek sprawdzania powtórzeń wykonuje się szybciej, gdy takich powtórzeń nie ma, natomiast dla board_vector trwa to dłużej, ponieważ duplikaty rzeczywiście występują.

-Jak reprezentacja będzie działać w algorytmach przeszukiwania.

Sądzę, że dzięki uniwersalności i braku przyjmowania nazw reprezentacji. Będzie bardziej przyjemna podczas używania w algorytmach przeszukiwania. Dużym plusem jest również przyjmowanie 4 różnych struktur co daje jeszcze większą uniwersalność.